Facultad de Ingeniería
Bernard Wand-Polak
Cuareim 1451
11.100 Montevideo, Uruguay
Tel. 2902 15 05 Fax 2908 13 70
www.ort.edu.uy

EVALUACIÓN	**Obligatorio**	**GRUPO**	TODOS	**FECHA**	Agosto 2026
MATERIA	Algoritmos y Estructuras de Datos 2 – Programación avanzada				
CARRERA					

CONDICIONES

- **Puntaje máximo:** 30 puntos
- **Puntaje mínimo:** 1 punto
- **Fecha de entrega:** 09/11/26 hasta las 21:00 horas en gestion.ort.edu.uy (max. 40 Mb en formato zip o rar)

Uso de material de apoyo y/o consulta

Inteligencia Artificial Generativa

- Seguir las pautas de los docentes: Se deben seguir las instrucciones específicas de los docentes sobre cómo utilizar la IA en cada curso.
- Citar correctamente las fuentes y usos de IA: Siempre que se utilice una herramienta de IA para generar contenido, se debe citar adecuadamente la fuente y la forma en que se utilizó. Se debe indicar:
  - la /las herramientas utilizadas y
  - el contexto de uso: por ejemplo, generación de código fuente, redacción de documentación, análisis de datos, corrección de errores, estilo, etc.
- Verificar el contenido generado por la IA: No todo el contenido generado por la IA es correcto o preciso. Es esencial que los estudiantes verifiquen la información antes de usarla.
- Ser responsables con el uso de la IA: Conocer los riesgos y desafíos, como la creación de "alucinaciones", los peligros para la privacidad, las cuestiones de propiedad intelectual, los sesgos inherentes y la producción de contenido falso
- El uso de herramientas de Inteligencia Artificial Generativa (IAG) puede emplearse como apoyo en el proceso de aprendizaje. Sin embargo, no sustituye el razonamiento crítico ni la elaboración personal. El estudiante es responsable de que el trabajo presentado refleje su propia comprensión, análisis y proceso cognitivo sobre el tema
- En caso de existir dudas sobre la autoría, plagio o uso no atribuido de IAG, el docente tendrá la opción de convocar al equipo de obligatorio a una defensa específica e individual sobre el tema
- La defensa es obligatoria y eliminatoria.
- La defensa será presencial salvo en los casos en que se habilite la defensa oral sincrónica en forma remota.

Defensa

Fecha de defensa: entre semana 14 y 16 del curso a confirmar con el docente.

La defensa es obligatoria y eliminatoria. El docente es quien definirá y comunicará la modalidad, y mecánica de defensa. La no presentación a la misma implica la pérdida de la totalidad de los puntos del Obligatorio.

IMPORTANTE:

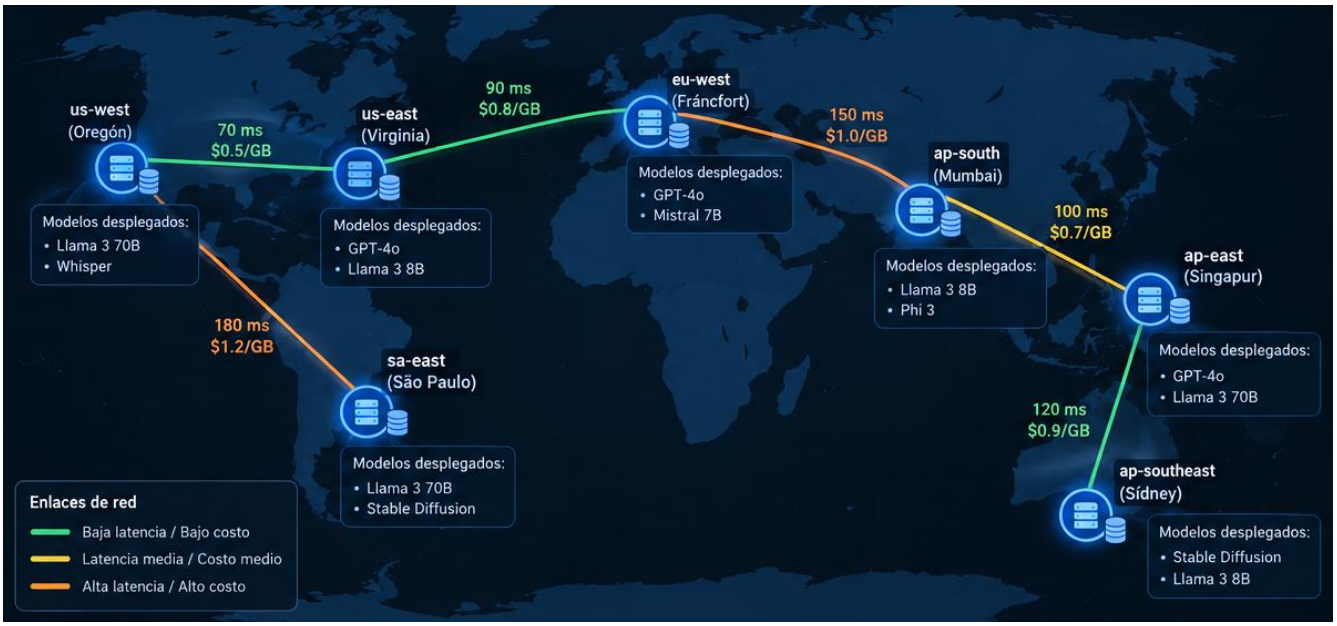
1) Inscribirse
2) Formar grupos de hasta 2 personas del mismo dictado
3) Subir el trabajo a Gestión antes de la hora indicada (ver hoja "RECORDATORIO" al final del documento)

Aquellos de ustedes que presenten alguna dificultad con su inscripción o tengan inconvenientes técnicos, por favor contactarse con el Coordinador o Coordinación adjunta antes de las 20:00hs. del día de la entrega, a través de los mails alamon@ort.edu.uy y rodríguez_mb@ort.edu.uy o telefónicamente al 29021505 - int 1156 u 1138

Introducción

Los modelos de aprendizaje profundo que se usan en producción no se ejecutan en un único lugar: se **despliegan** en varias regiones de datacenters distribuidas por el mundo, y los pesos de cada modelo se replican entre esas regiones a través de enlaces de red. Cada enlace tiene dos costos que compiten entre sí: la **latencia** (cuánto demora un paquete en atravesarlo, en milisegundos) y el **costo de transferencia** de datos entre regiones (el egreso de datos, en centavos de dólar por GB transferido).

Se desea implementar un programa para gestionar de forma eficiente una **red global de inferencia**: por un lado, el registro y la consulta de los despliegues de modelos, y por otro la topología de regiones y enlaces, para poder responder por dónde conviene mover los datos según se quiera minimizar latencia o costo.



Todas las operaciones deberán devolver una instancia de la clase Retorno. Dicha clase contiene:

- Un **resultado**, que especifica si la operación se pudo realizar correctamente (OK), o si ocurrió algún error (según el número de error).
- Un **valor entero**, para las operaciones que retornen un número entero.
- Un **valor String**, para las operaciones que retornen un String, o un valor más complejo, por ejemplo, un listado, el cual será formateado según lo indicado en los ejemplos.

```
public class Retorno {
     public enum Resultado {OK, ERROR_1, ERROR_2, ERROR_3, ERROR_4,
     ERROR_5, ERROR_6, ERROR_7, ERROR_8, NO_IMPLEMENTADA};
     public int valorEntero;
     public String valorString;
     public Resultado resultado;
}
```

Se provee: una interfaz llamada **Sistema**, la cual no podrá ser modificada en ningún sentido y una clase **ImplementacionSistema** que la implementa, donde su equipo deberá completar la implementación de las operaciones solicitadas.

Consideraciones:

- La clase **ImplementacionSistema** NO PODRÁ SER UN SINGLETON. Debe ser una clase **instanciable**.
- Pueden definirse tipos de datos (clases) auxiliares.
- Se provee un proyecto base que debe ser utilizado, utilizar el proyecto es obligatorio.

Funcionalidades

01 - Inicializar Sistema

```
Retorno inicializarSistema (int maxRegiones);
```

Descripción: Inicializa las estructuras necesarias para representar el sistema especificado, capaz de registrar como máximo la cantidad maxRegiones de regiones de datacenter diferentes en el sistema. Luego de esta operación el sistema queda vacío: sin despliegues, sin regiones y sin enlaces registrados.

Restricción de eficiencia: no tiene.

Retornos posibles	
OK	El sistema pudo ser inicializado exitosamente.
ERROR	1. Si `maxRegiones` es menor o igual a 3.
NO_IMPLEMENTADA	Cuando aún no se implementó. Es el tipo de retorno por defecto.

02 - Registrar despliegue

```
Retorno registrarDespliegue(String id, String codigo, String descripcion, boolean
cuantizado, Modalidad modalidad);
```

Descripción: Registra el despliegue de un modelo con sus datos. El id es el identificador interno del despliegue y el código identifica la versión exacta de los pesos publicados (el checkpoint); ambos son únicos, el parámetro cuantizado indica si el modelo se sirve con pesos de precisión reducida, mientras que modalidad indica el tipo de modelo.

Restricción de eficiencia: Esta operación deberá realizarse en orden O(log n) promedio, siendo n la cantidad total de despliegues.

Retornos posibles	
OK	El despliegue fue registrado exitosamente.

ERROR	1. Si id, código o descripción son vacíos o null, o modalidad es null. 2. Si código no tiene el formato válido. 3. Si ya existe un despliegue registrado con ese id. 4. Si ya existe un despliegue registrado con ese código.
NO_IMPLEMENTADA	Cuando aún no se implementó.

Restricción: (**Investigación**) se requiere el uso de expresiones regulares para lograr validar el formato del código. El formato para validar es **AAA-BBB-CCCC**, donde:

- **AAA** es la familia del modelo: exactamente 3 letras mayúsculas (por ejemplo LLM , VIS , AUD).
- **BBB** es el tamaño del modelo en miles de millones de parámetros: exactamente 3 dígitos.
- **CCCC** es el identificador de build: exactamente 4 caracteres alfanuméricos (solo letras, mayúsculas o minúsculas, y números).

Las modalidades posibles son:

- Texto
- Visión
- Audio
- Multimodal
- Otros

Ejemplos:

Resultado	Operación
OK	`registrarDespliegue("DEP-0001", "LLM-070-A1B2", "Modelo de texto 70B en producción", true, Modalidad.TEXTO)`
ERROR_1	`registrarDespliegue("DEP-0001", "LLM-070-A1B2", "", false, Modalidad.OTROS)`
ERROR_1	`registrarDespliegue("DEP-0001", "LLM-070-A1B2", " ", false, Modalidad.OTROS)`
ERROR_1	`registrarDespliegue(null, "LLM-070-A1B2", "Modelo de texto 70B", false, Modalidad.OTROS)`
ERROR_2	`registrarDespliegue("DEP-0001", "llm-070-A1B2", "Modelo de texto 70B", false, Modalidad.OTROS)`
ERROR_2	`registrarDespliegue("DEP-0001", "LLM-0B0-A1_2", "Modelo de texto 70B", false, Modalidad.OTROS)`

03 - Buscar despliegue por id

```
Retorno buscarDesplieguePorId(String id);
```

Descripción: Retorna en valorString los datos del despliegue con el formato "id;codigo;descripcion;cuantizado;modalidad". Además, en el campo valorEntero del objeto retorno deberá devolver la cantidad de elementos que recorrió durante la búsqueda en la estructura utilizada.

Restricción de eficiencia: Esta operación deberá realizarse en orden O(log n) promedio, siendo n la cantidad total de despliegues.

Retornos posibles	
OK	Si el despliegue se encontró. Retorna en valorString los datos del despliegue. Retorna en valorEntero la cantidad de elementos recorridos durante la búsqueda.
ERROR	1. Si el id es vacío o null. 2. Si no existe un despliegue registrado con ese id.
NO_IMPLEMENTADA	Cuando aún no se implementó.

Formato de retorno del valorString:

```
id;codigo;descripcion;cuantizado;modalidad
```

Por ejemplo, valorString del retorno en una consulta válida:

```
DEP-0001;LLM-070-A1B2;Modelo de texto 70B en producción;true;Texto
```

04 - Listar despliegues por id ascendente

```
Retorno listarDesplieguesPorIdAscendente();
```

Descripción: Retorna en valorString los datos de todos los despliegues registrados, ordenados por id en forma creciente separados por un |.

Restricción de eficiencia: Esta operación deberá realizarse en orden O(n), siendo n la cantidad total de despliegues.

Retornos posibles	
OK	Retornando el listado de despliegues en valorString.
ERROR	No hay errores posibles.
NO_IMPLEMENTADA	Cuando aún no se implementó.

Formato de retorno del valor String:

```
id1;codigo1;descripcion1;cuantizado1;modalidad1|
id2;codigo2;descripcion2;cuantizado2;modalidad2
```

Nota: Se debe cumplir que id1 es lexicográficamente menor a id2.

Por ejemplo:

```
DEP-0001;LLM-070-A1B2;Modelo de texto 70B;true;Texto|DEP-0002;VIS-
004-C3D4;Modelo de visión 4B;false;Visión
```

05 - Listar despliegues por id descendente

```
Retorno listarDesplieguesPorIdDescendente();
```

Descripción: Retorna en valorString los datos de todos los despliegues registrados, ordenados por id en forma decreciente.

Restricción de eficiencia: Esta operación deberá realizarse en orden O(n), siendo n la cantidad total de despliegues.

Retornos posibles	
OK	Retornando el listado de despliegues en valorString.
ERROR	No hay errores posibles.
NO_IMPLEMENTADA	Cuando aún no se implementó.

Nota: Se debe cumplir que id1 es lexicográficamente mayor a id2.

Por ejemplo:

```
DEP-0002;VIS-004-C3D4;Modelo de visión 4B;false;Visión|DEP-0001;LLM-070-A1B2;Modelo de texto 70B;true;Texto
```

06 - Buscar despliegue por código

```
Retorno buscarDesplieguePorCodigo(String codigo);
```

Descripción: retorna en valorString los datos del despliegue con el formato "id;codigo;descripcion;cuantizado;modalidad". Además, en el campo valorEntero del objeto retorno deberá devolver la cantidad de elementos que recorrió durante la búsqueda en la estructura utilizada.

Restricción de eficiencia: Esta operación deberá realizarse en orden O(log n) promedio, siendo n la cantidad total de despliegues.

Retornos posibles	
OK	Si el despliegue se encontró. Retorna en valorString los datos del despliegue. Retorna en valorEntero la cantidad de elementos recorridos durante la búsqueda.
ERROR	1. Si el código es vacío o null. 2. Si no existe un despliegue con ese código.
NO_IMPLEMENTADA	Cuando aún no se implementó.

07 - Listar despliegues por código ascendente

```
Retorno listarDesplieguesPorCodigoAscendente();
```

Descripción: Retorna en valorString los datos de todos los despliegues registrados, ordenados por código en forma creciente.

Restricción de eficiencia: Esta operación deberá realizarse en orden O(n), siendo n la cantidad total de despliegues.

Retornos posibles	
OK	Retornando el listado de despliegues en valorString.
ERROR	No hay errores posibles.
NO_IMPLEMENTADA	Cuando aún no se implementó.

Nota: Se debe cumplir que codigo1 es lexicográficamente menor a codigo2.

Por ejemplo:

```
DEP-0002;AUD-002-C3D4;Modelo de audio 2B;false;Audio|DEP-0001;LLM-070-A1B2;Modelo de texto 70B;true;Texto
```

08 - Listar despliegues por modalidad

```
Retorno listarDesplieguesPorModalidad(Modalidad unaModalidad);
```

Descripción: Retorna en valorString los datos de todos los despliegues registrados con esa modalidad, ordenados de forma creciente por código.

Restricción de eficiencia: Esta operación deberá realizarse en orden O(k), siendo k la cantidad de despliegues con dicha modalidad.

Retornos posibles	
OK	Se pudo listar los despliegues que pertenecen a esa modalidad correctamente.
ERROR	No hay errores posibles.
NO_IMPLEMENTADA	Cuando aún no se implementó.

Formato de retorno del valor String:

```
id1;codigo1;descripcion1;cuantizado1;modalidad|id2;codigo2;descripcion2;cuantizado2;modalidad
```

09 - Registrar región

```
Retorno registrarRegion(String codigo, String nombre, String pais, String zona);
```

Descripción: Registra una región de datacenters en el sistema con el código, nombre, país y zona de disponibilidad indicados. El código es el identificador único (por ejemplo us-east-1). Ninguno de los parámetros puede ser vacío ni null. Esta operación no tiene restricciones de eficiencia.

Retornos posibles	
OK	Si la región fue registrada exitosamente.
ERROR	1. Si en el sistema ya hay registradas maxRegiones regiones. 2. Si código, nombre, país o zona son vacíos o null. 3. Si ya existe una región con ese código.
NO_IMPLEMENTADA	Cuando aún no se implementó.

10 - Registrar enlace

```
Retorno registrarEnlace(String codigoOrigen, String codigoDestino, int latencia,
int costo);
```

Descripción: Registra un enlace de red en el sistema desde la región con código codigoOrigen hacia la región con código codigoDestino, indicando la latencia en milisegundos y el costo de transferencia en centavos de dólar por GB. Esta operación no tiene restricciones de eficiencia.

Retornos posibles	
OK	Si el enlace fue registrado exitosamente.
ERROR	1. Si alguno de los parámetros String es vacío o null. 2. Si no existe la región de origen. 3. Si no existe la región de destino. 4. Si la latencia es menor o igual a cero. 5. Si el costo es menor o igual a cero. 6. Si ya existe un enlace entre el origen y el destino.
NO_IMPLEMENTADA	Cuando aún no se implementó.

Nota**:** se considera que los enlaces son navegables en ambos sentidos. O sea que, si se registra un enlace de la región A a la B, debe quedar definido también desde B a A.

11 - Red de regiones por cantidad de saltos

`Retorno` **`redRegionesPorCantidadDeSaltos`**`(String codigoOrigen, int cantidad);`

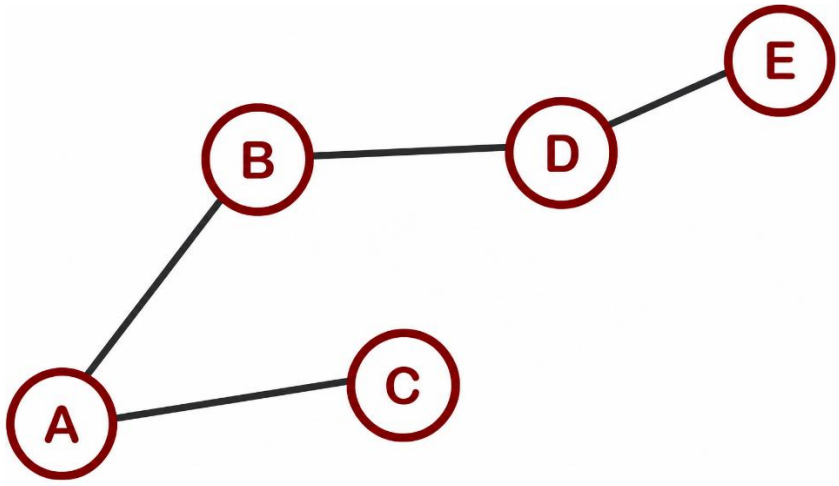
Descripción: Dada una región de origen se debe retornar en el valorString los datos de las regiones (ordenadas por código creciente y sin repetidos) a las que se pueda llegar realizando hasta la cantidad de enlaces indicada por el parámetro. La región de origen forma parte del resultado, ya que se llega a ella con cero saltos. Esta operación no tiene restricciones de eficiencia.

Retornos posibles	
OK	Retorna en valorString los datos de las regiones a las que se pueda llegar con hasta "cantidad" de enlaces desde la región de origen ordenadas de forma creciente por código.
ERROR	1. Si la cantidad es menor que cero. 2. Si el código es vacío o null. 3. Si la región no está registrada en el sistema.
NO_IMPLEMENTADA	Cuando aún no se implementó.

Formato de retorno del valor String:

`codigo1;nombre1;pais1;zona1|codigo2;nombre2;pais2;zona2`

Nota: Se debe cumplir que `codigo1` es lexicográficamente menor a `codigo2`.



12 – Ruta de menor latencia

```
Retorno rutaMenorLatencia (String codigoOrigen, String codigoDestino);
```

Descripción: retorna la ruta más eficiente en cuanto a latencia que se podría recorrer para ir de la región de origen a la región de destino. Esta operación no tiene restricciones de eficiencia.

Retornos posibles	
OK	Si la ruta pudo ser calculada exitosamente. Retorna en valorEntero la latencia total acumulada de la ruta, en milisegundos. Retorna en valorString la ruta desde la región de origen a la región de destino, ambas incluidas.
ERROR	1. Si alguno de los parámetros es vacío o null. 2. Si no existe la región de origen. 3. Si no existe la región de destino. 4. Si no hay **ruta** entre el origen y el destino.
NO_IMPLEMENTADA	Cuando aún no se implementó.

Formato de retorno del valor String:
`codigoOrigen;nombreOrigen;paisOrigen;zonaOrigen|codigo1;nombre1;pais1;zona1|...|codigoDestino;nombreDestino;paisDestino;zonaDestino`

Nota: Notar que el error 4 es cuando **no existe una ruta posible**, puede existir una ruta sin un enlace directo. Si el origen y el destino son la misma región, se retorna OK con valorEntero igual a 0 y valorString con los datos de esa única región

13 – Ruta de menor costo

```
Retorno rutaMenorCosto (String codigoOrigen, String codigoDestino);
```

Descripción: Retorna la ruta más eficiente en cuanto a costo de transferencia que se podría recorrer para ir de la región de origen a la región de destino. Esta operación no tiene restricciones de eficiencia. Nota: La ruta de menor costo no es necesariamente la misma que la de menor latencia, ni la que atraviesa menos enlaces.

Retornos posibles	
OK	Si la ruta pudo ser calculada exitosamente. Retorna en valorEntero el costo total acumulado de la ruta. Retorna en valorString la ruta desde la región de origen a la región de destino, ambas incluidas.
ERROR	1. Si alguno de los parámetros es vacío o null. 2. Si no existe la región de origen. 3. Si no existe la región de destino. 4. Si no hay **ruta** entre el origen y el destino.
NO_IMPLEMENTADA	Cuando aún no se implementó.

Información importante

- **Se deberán respetar los formatos de retorno dados para las operaciones que devuelven datos. Está terminantemente prohibido el uso de clases de Java tales como ArrayList, HashMap, el uso de estas clases implica la pérdida de todos los puntos de la operación en la que se utilizaron.**
- Ninguna de las operaciones debe imprimir nada en consola.
- El sistema no debe requerir ningún tipo de interacción con el usuario por consola.
- Es obligación del estudiante mantenerse al tanto de las aclaraciones que se realicen en clase o a través del foro de aulas.
- Es necesaria una selección adecuada de las estructuras para modelar el problema y la eficiencia en cada una de las operaciones. Deberá aplicar la metodología vista en el curso, resolver el obligatorio (incluso pasando las pruebas funcionales) sin cumplir los órdenes puede implicar la pérdida del total de los puntos de la operación.
- El proyecto será implementado en lenguaje JAVA en el ide IntelliJ IDEA, ya se encuentra disponible en el sitio de la materia en aulas.ort.edu.uy. La interfaz Sistema no se puede modificar. El uso del proyecto provisto junto a la letra es obligatorio.
- El proyecto entregado debe compilar y ejecutar correctamente en IntelliJ IDEA. De no ser así implica la pérdida de todos los puntos del obligatorio. Si no se logra implementar una operación se debe dejar en un estado consistente o como “No implementada”.
- No se contestarán dudas sobre el obligatorio en las 48 horas previas a la entrega.
- El equipo debe entregar un conjunto de pruebas unitarias para evidenciar una correcta estrategia de testing aplicada al proyecto.
- Una semana antes de la entrega la cátedra dejará disponible un conjunto de pruebas (no exhaustivas) para todas las operaciones para que puedan ajustar las desviaciones con las pruebas propias.
- Defensa: la defensa del obligatorio será en fecha y formato a confirmar por el docente. Se publicará en aulas la fecha y horario específico con antelación para cada grupo. La no asistencia a la defensa implica la pérdida de todos los puntos.
- Considere que superar las pruebas no garantiza que la operación se haya resuelto correctamente.
- **Entrega (Se debe subir un único zip o rar con):**
    - **Proyecto con las operaciones implementadas respetando las buenas prácticas y estándares vistos en el curso.**
    - **La implementación debe respetar los órdenes solicitados.**
    - Incluir en la parte superior de la clase ImplementacionSistema los nombres y números de estudiante del equipo.
    - Conjunto de pruebas unitarias realizadas por el equipo.
    - Documentación: Entregar un documento PDF con una carátula con información de la materia, evaluación y del equipo. En el cuerpo del documento agregar una descripción de las estructuras utilizadas y una justificación del cumplimiento de los órdenes solicitados.
- El ABB utilizado se debe implementar de forma genérica, como también Pila, Lista y Cola si se utilizan.
- El Grafo utilizado no es necesario implementarlo de forma genérica.
- Ninguno de los listados (que se cargan en el valorString) termina en |, o sea al agregar el último elemento no se debe agregar el |.

Facultad de Ingeniería
Bernard Wand-Polak
Cuareim 1451
11.100 Montevideo, Uruguay
Tel. 2902 15 05 Fax 2908 13 70
www.ort.edu.uy

RECORDATORIO: IMPORTANTE PARA LA ENTREGA

La entrega de los obligatorios será en formato digital online, a excepción de algunas materias que se entregarán en Bedelía y en ese caso recibirá información específica en el dictado de la misma.

Los principales aspectos a destacar sobre la **entrega online de obligatorios** son:

1. Ingresá al sistema de Gestión.
2. En el menú, seleccioná el ítem “Evaluaciones” y la instancia de evaluación correspondiente, que figura bajo el título “Inscripto”.
3. Para iniciar la entrega hacé clic en el ícono: 
4. Ingresá el número de estudiante de cada uno de los integrantes y hacé clic en “Agregar”. El sistema confirmará que los integrantes estén inscriptos al obligatorio y, de ser así, mostrará el nombre y la fotografía de cada uno de ellos. Una vez agregados todos los integrantes, hacé clic en “Crear equipo”.

   Cualquier integrante podrá:

   • Modificar la integración del equipo.

   • Subir el archivo de la entrega.

5. Seleccioná el archivo que deseás entregar. Verificá el nombre del archivo que aparecerá en la pantalla y hacé clic en “Subir” para iniciar la entrega. Cada equipo (hasta 2 estudiantes) debe entregar **un único archivo en formato zip o rar** (los documentos de texto deben ser pdf, y deben ir dentro del zip o rar). El archivo a subir debe tener **un tamaño máximo de 40mb**

   Cuando el archivo quede subido, se mostrará el nombre generado por el sistema (1), el tamaño y la fecha en que fue subido.

6. El sistema enviará un e-mail a todos los integrantes del equipo informando los detalles del archivo entregado y confirmando que la entrega fue realizada correctamente.
7. Podés cerrar la pestaña de entrega y continuar utilizando Gestión o salir del sistema.
8. La **hora tope para subir el archivo será las 21:00** del día fijado para la entrega.
9. La entrega se podrá realizar desde cualquier lugar (ej. hogar del estudiante, laboratorios de la Universidad, etc).
10. Aquellos de ustedes que presenten alguna dificultad con su inscripción o tengan inconvenientes técnicos, por favor contactarse con la Coordinadora o Coordinación adjunta antes de las 20:00hs. del día de la entrega, a través de los mails, alamon@ort.edu.uy o rodríguez_mb@ort.edu.uy; o telefónicamente al 29021505 - int 1156 u 1138